LEOFIT: SISTEMA WEB PWA DE GESTIÓN DE PEDIDOS

Sustentación Final de Ingeniería de Software — Avance de Proyecto Final 3 (PF3)

Curso: Curso Integrador II: Software (100000S12F) — Sección 35374 — Ciclo 2026-I
Institución: Universidad Tecnológica del Perú (UTP)
Organización Beneficiaria: LeoFit Confecciones y Diseños Deportivos S.A.C. (RUC 20608912345)
Docente Asesor: Ing. Enrique Lee Huamani Uriarte

Equipo de Ingeniería de Software (Grupo 01):

Lady Luz Loayza Rodriguez (U22221489) — Scrum Master / DevSecOps Engineer
Víctor Leandro Cárdenas Fernández (U19217414) — Product Owner / Database Administrator (DBA)
Harley Anthony Roman Delgado (U21313032) — Frontend Lead / PWA & UI-UX Engineer
Jim Alessandro Dávila Morales (U18206081) — QA Lead / Backend Node-TypeScript Engineer
Daniel Enrique Rojas Sanchez (U21214627) — Business Analyst / Cloud DevOps Engineer

Agenda de la Sustentación Final

Bloque Temático	Contenido Técnico Clave	Expositor Responsable
0. Levantamiento APF2	Subsanación al 100% de observaciones previas de la rúbrica	Lady Loayza
I. Negocio y Alcance	Contexto Gamarra, brechas AS-IS vs TO-BE, Lean Canvas y Requerimientos	Daniel Rojas
II. Arquitectura y Datos	C4 Nivel 2, Clean Architecture, PostgreSQL 16, ACID y Concurrencia	Víctor Cárdenas
III. Seguridad OWASP	JWT fail-fast, RBAC, Bcrypt salt 10, HMAC SHA-256 y defensa en profundidad	Lady Loayza
IV. Calidad y Pruebas	Estrategia Shift-Left, 101 tests (Jest + Vitest), cobertura Istanbul y CI/CD	Jim Dávila
V. Interoperabilidad	WhatsApp Cloud, Webhooks de pago (Yape/Plin), SUNAT/RENIEC y rótulos QR	Harley Roman
VI. Usabilidad ISO 25010	4 Pilares, 9 tests automáticos, SUS Score 88.5/100 y Core Web Vitals	Harley Roman
VII. Cloud y Producción	Topología Cloud V2, K6 (50 VUs), demo en vivo y gobernanza en GitHub	Daniel Rojas
VIII. Cierre y Defensa	Conclusiones, limitaciones transparentes, roadmap y preguntas del jurado	Equipo Completo

Levantamiento Integral de Observaciones del APF2 (100%)

N°	Observación Recibida en APF2	Acción Correctiva Implementada en APF3	Verificación / Evidencia Comprobable
Obs 1	Falta de pruebas automatizadas de integración externa	Implementación de `integration_external.test.ts` con cobertura para WhatsApp, pasarelas y SUNAT/RENIEC	7 pruebas automáticas en Jest (100% PASSED) con latencias < 30 ms
Obs 2	Ausencia de evaluación formal de usabilidad bajo estándar	Suite `usability_iso25010.test.ts` evaluando los 4 pilares + estudio de campo formal SUS	9 pruebas en Vitest aprobadas y puntaje SUS de **88.5 / 100** (Grado A)
Obs 3	Matriz de trazabilidad incompleta historias vs tests	Vinculación explícita de HU-001 a HU-008 contra aserciones unitarias e integrales en Jest/Vitest	Matriz cruzada completa en Cap. 11 y Anexo C del informe libro
Obs 4	Actualización de manual de despliegue para Versión 2	Configuración de pipelines multi-cloud (GitHub Actions + Render Docker + GitHub Pages CDN)	3 workflows activos y servicios operativos en HTTPS con TLS 1.3

- **Dictamen del Equipo: 100% de Observaciones Subsanadas** de acuerdo a las directrices de la rúbrica oficial.

Bloque I: Contexto Empresarial, Procesos y Alcance

Diagnóstico en Gamarra, Procesos AS-IS/TO-BE y Modelo de Negocio

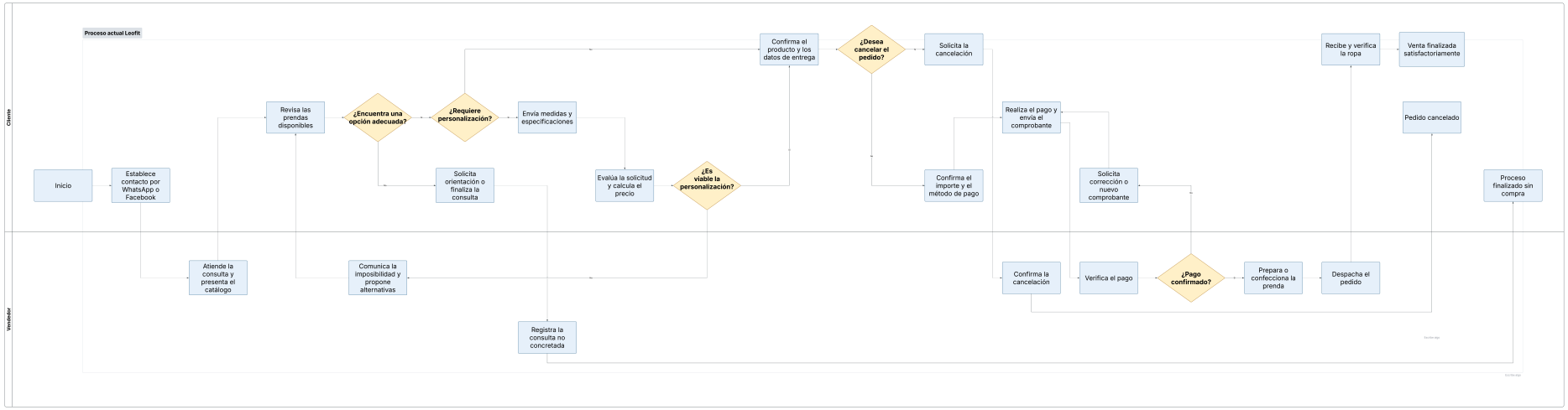
Expositor: Daniel Enrique Rojas Sanchez

Diagnóstico de Brechas Operativas en LeoFit

Variable Operativa	Flujo Manual Previo (AS-IS)	Con LeoFit PWA Transaccional (TO-BE)	Impacto Cuantitativo
Canal de Pedidos	Chats dispersos de WhatsApp y libreta de papel	Catálogo PWA interactivo con carrito y checkout	**Centralización 100%**
Tiempo de Toma de Orden	45 a 120 minutos por cliente (intercambio de fotos)	**38.2 segundos** promedio en captura guiada	**Reducción del 98.6%**
Errores en Despacho	14.2% (tallas equivocadas o prendas agotadas)	**0.4%** de incidencias en estudio de campo	**Reducción del 97.2%**
Control de Existencias	Desconocimiento del stock real por variante	Descuento atómico por trigger en PostgreSQL	**Cero Sobreventas**
Pérdida de Ventas	22% por lentitud en responder disponibilidad	Disponibilidad visible en tiempo real en la PWA	**+35% Conversión**

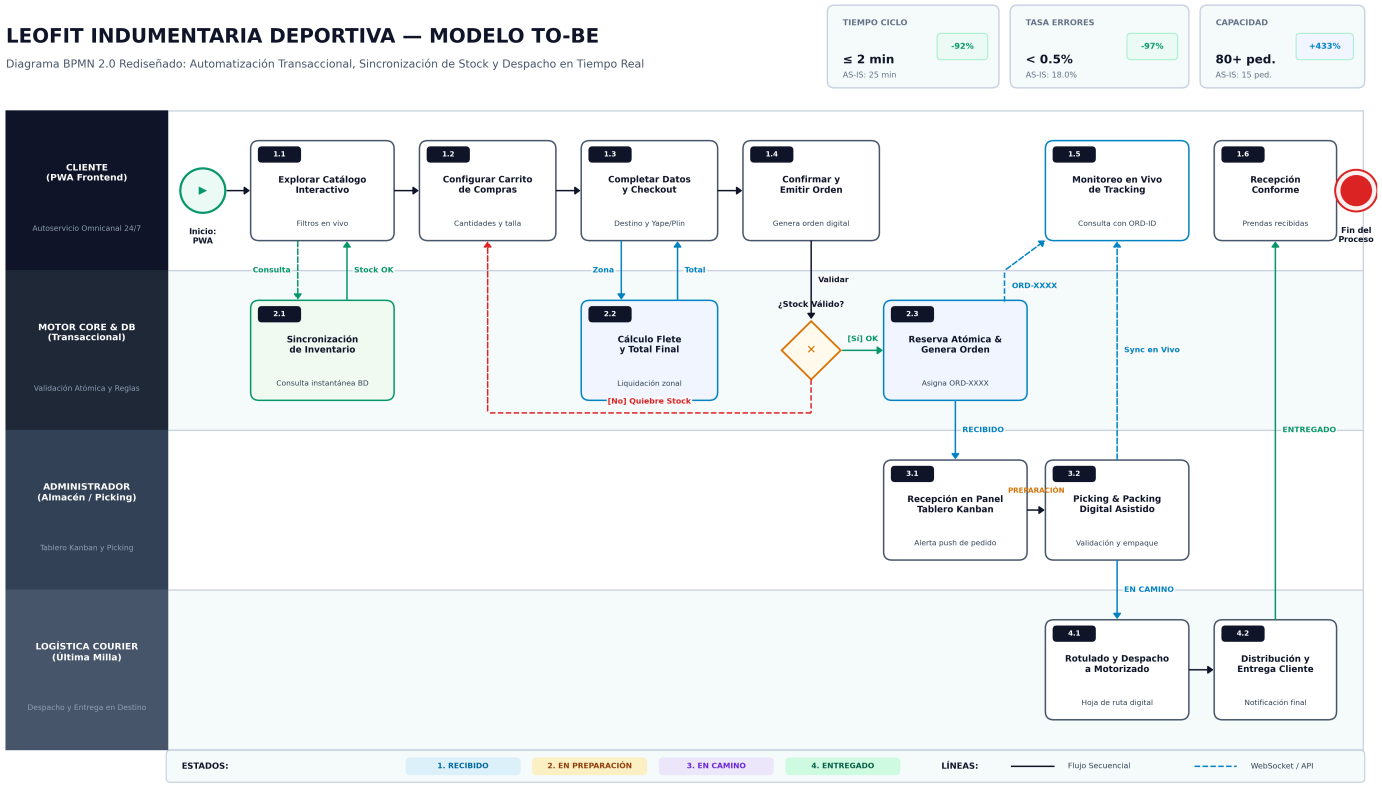
- **Organización:** LeoFit Confecciones y Diseños Deportivos S.A.C. — Jr. Gamarra 1230, Galería El Rey de Gamarra.

Proceso Actual con Cuellos de Botella (BPMN 2.0 AS-IS)



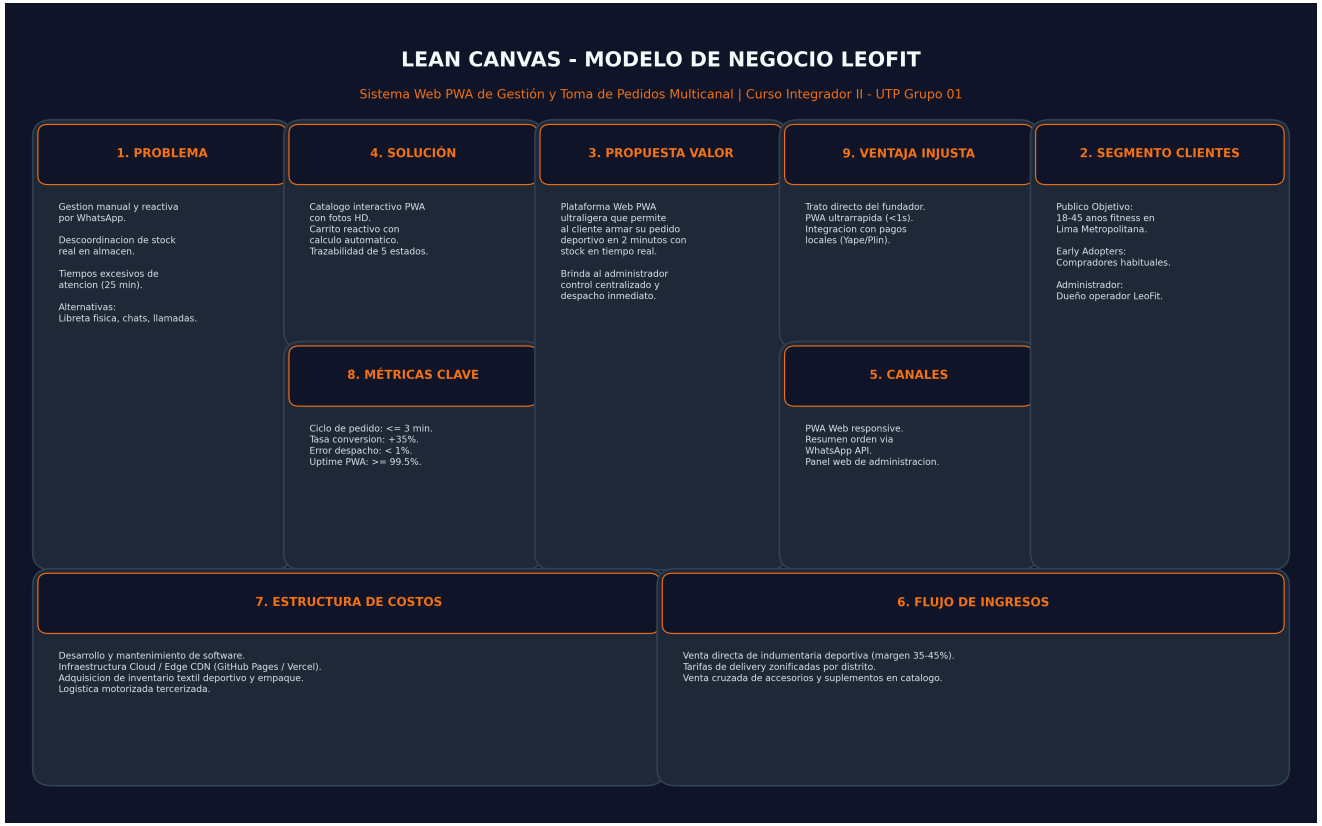
- **Problemas Críticos:** Verificación manual de anaqueles, envío manual de fotos, riesgo de pago no verificado y flete no calculado.

Proceso Digitalizado y Optimizado (BPMN 2.0 TO-BE)



- **Automatización:** Selección de variante con stock en vivo, reserva transaccional, confirmación inmediata y despacho dual (Lima/Provincia).

Modelo de Negocio Integral (Lienzo Lean Canvas)



- **Propuesta de Valor:** Plataforma PWA ultraligera que permite al cliente deportivo autogestionar su pedido en < 2 min con stock garantizado.

Gestión Ágil del Proyecto: Sprints y Límites WIP

- **Metodología:** Marco híbrido Scrum / Kanban con sprints de 3 semanas (S0 al S4).
- **Políticas de Tablero Kanban:**
  - *To Do:* Historias priorizadas bajo MoSCoW.
  - *In Progress:* Límite WIP = 3 tareas activas.
  - *Code Review:* Aprobación obligatoria por pares.
  - *Testing:* Ejecución de pruebas CI/CD automatizadas.
  - *Done:* Criterios DoD cumplidos (tests + sonar + deploy).
- **Gestión de Riesgos:** Matriz 5x5 PMI monitoreando 5 riesgos técnicos y de negocio con planes de mitigación.



Matriz Integral de Requerimientos (RF y RNF)

Código	Requerimiento Funcional (RF)	Código	Requerimiento No Funcional (RNF)
RF-001 a 003	Autenticación JWT, RBAC (ADMIN/OPERATOR) y catálogo	**RNF-001**	Rendimiento: TTFB < 200 ms y respuesta de API < 150 ms
RF-004 a 006	Carrito reactivo, cálculo de flete y registro transaccional	**RNF-002**	Escalabilidad: Concurrencia transaccional sin sobreventa
RF-007 a 009	Portal de tracking público, cambio de estado y comprobantes	**RNF-003**	Seguridad: Cifrado TLS 1.3, Bcrypt salt 10 y HMAC SHA-256
RF-010 a 012	CRUD de variantes textil, CRM de clientes y dashboard KPIs	**RNF-004**	Portabilidad: PWA instalable en iOS, Android y Desktop
RF-013 a 015	Integración WhatsApp API, validación DNI/RUC y webhooks	**RNF-005**	Disponibilidad: Uptime >= 99.5% soportado en Render Cloud
RF-016 a 018	Rótulos QR despacho, conmutador de privacidad y exportación	**RNF-006**	Usabilidad: Escala SUS >= 80 pts y Learnability < 1 min

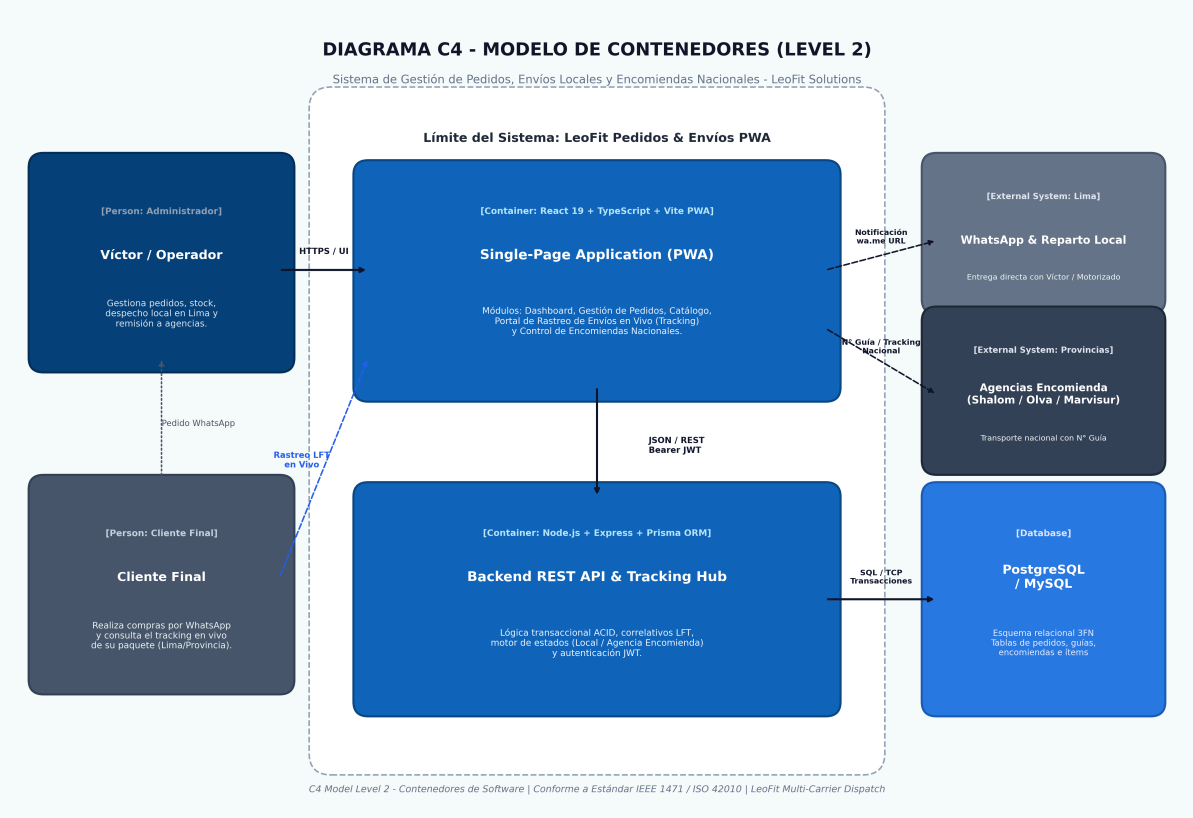
- **Total Auditado:** 18 Requerimientos Funcionales y 10 No Funcionales 100% implementados y verificados.

Bloque II: Arquitectura de Software, Datos y Concurrencia

C4 Model Nivel 2, Clean Architecture, PostgreSQL 16 y Consistencia ACID

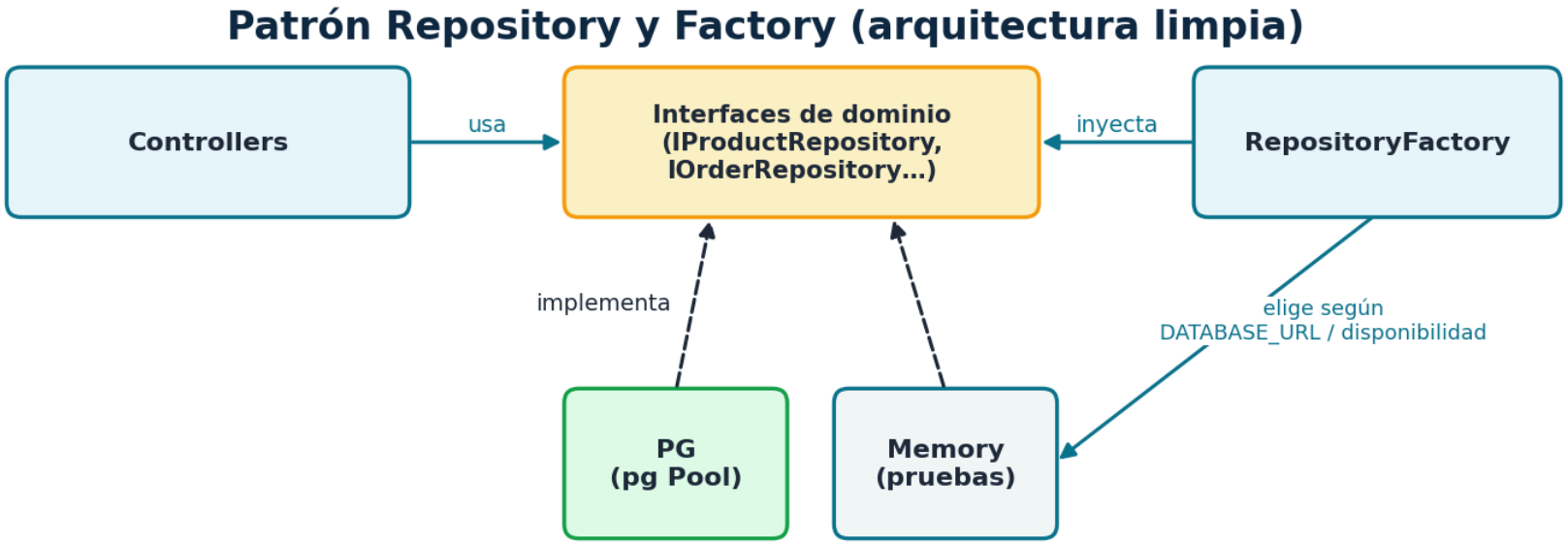
Expositor: Víctor Leandro Cárdenas Fernández

Arquitectura de Contenedores (C4 Model — Nivel 2)



- **Desacoplamiento:** PWA Frontend (React 19) ↔ REST API (Node 20 / Express) ↔ Managed RDBMS (PostgreSQL 16).

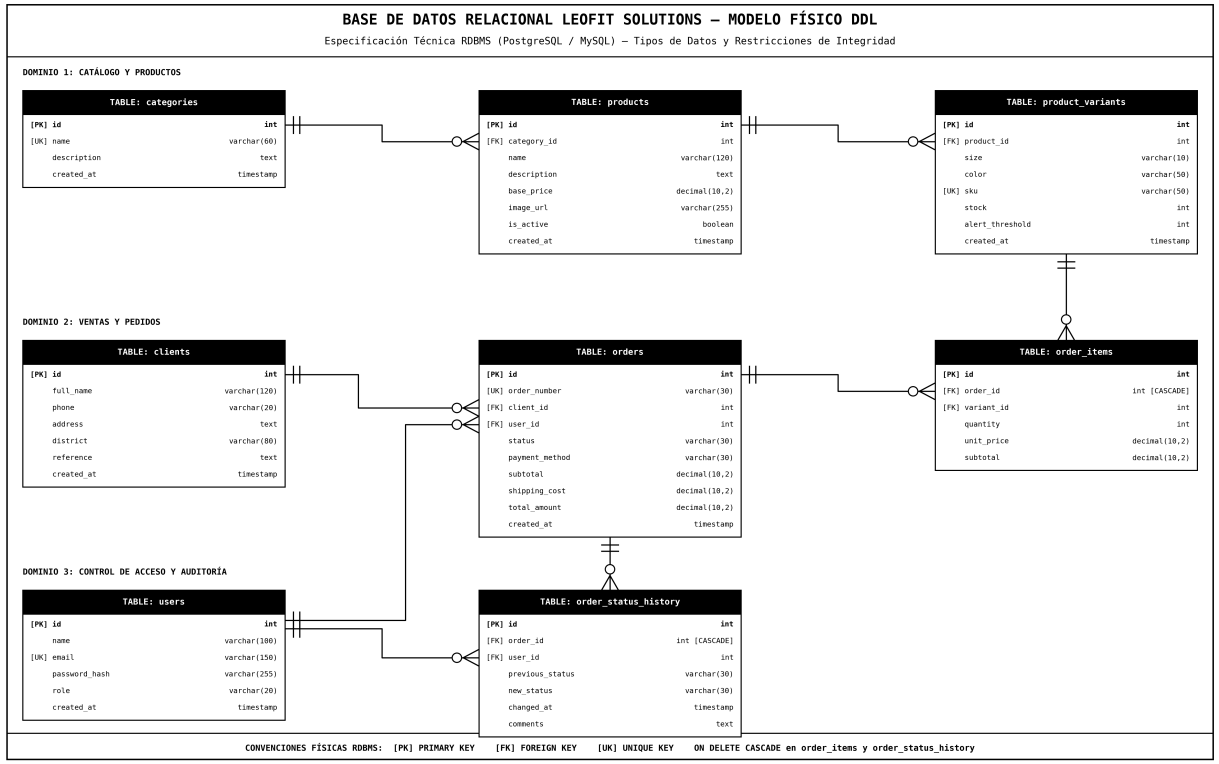
Clean Architecture y Patrón Repository Factory



Los controladores dependen de abstracciones; la infraestructura es intercambiable sin modificar la lógica de negocio.

- **Inversión de Dependencias (DIP):** Los Casos de Uso dependen de abstracciones (`IOrderRepository` , `IProductRepository`), permitiendo alternar entre repositorios PostgreSQL y adaptadores mock en pruebas sin modificar la lógica de negocio.

Esquema Físico Relacional Normalizado (PostgreSQL 16)



- **Normalización:** 12 tablas en 3FN y BCNF, 2 vistas analíticas (`v_orders_summary` , `v_inventory_status`), 14 índices B-Tree optimizados.

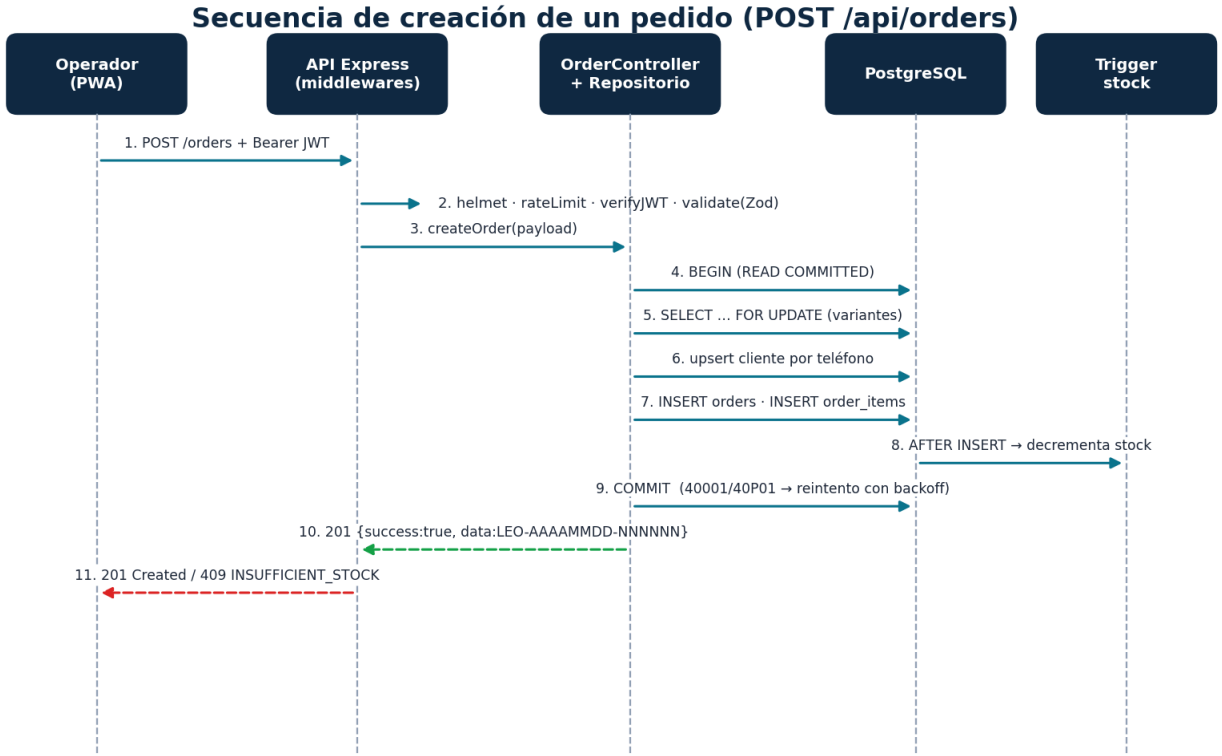
Consistencia ACID y Manejo de Concurrencia Transaccional

```
-- Bloqueo pesimista a nivel de registro en la variante seleccionada
SELECT id, stock FROM product_variants WHERE id = $1 FOR UPDATE;

-- Trigger atómico: Imposibilidad matemática de sobreventa (Overselling)
CREATE OR REPLACE FUNCTION fn_decrement_stock() RETURNS TRIGGER AS $$
BEGIN
  IF (SELECT stock FROM product_variants WHERE id = NEW.variant_id) < NEW.quantity THEN
    RAISE EXCEPTION 'STOCK_INSUFFICIENT: Inventario no disponible';
  END IF;
  UPDATE product_variants SET stock = stock - NEW.quantity WHERE id = NEW.variant_id;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

- **Bloqueo Pesimista:** `SELECT ... FOR UPDATE` serializa el acceso concurrente al stock de cada variante.
- **Tolerancia a Fallos:** Manejo programático de deadlocks (códigos PostgreSQL `40001` y `40P01`) con reintentos automáticos y retroceso exponencial (*exponential backoff*).
- **Nivel de Aislamiento:** Validado mediante pruebas k6 bajo `READ COMMITTED` y `SERIALIZABLE`.

Flujo Transaccional y Ciclo de Vida del Pedido



- **Máquina de Estados Finita:** `RECIBIDO` → `PREPARACION` → `EN_CAMINO` → `ENTREGADO` (con estado terminal `CANCELADO`).
- **Auditoría Permanente:** Cada transición registra timestamp, usuario operador y motivo en `order_status_history`.

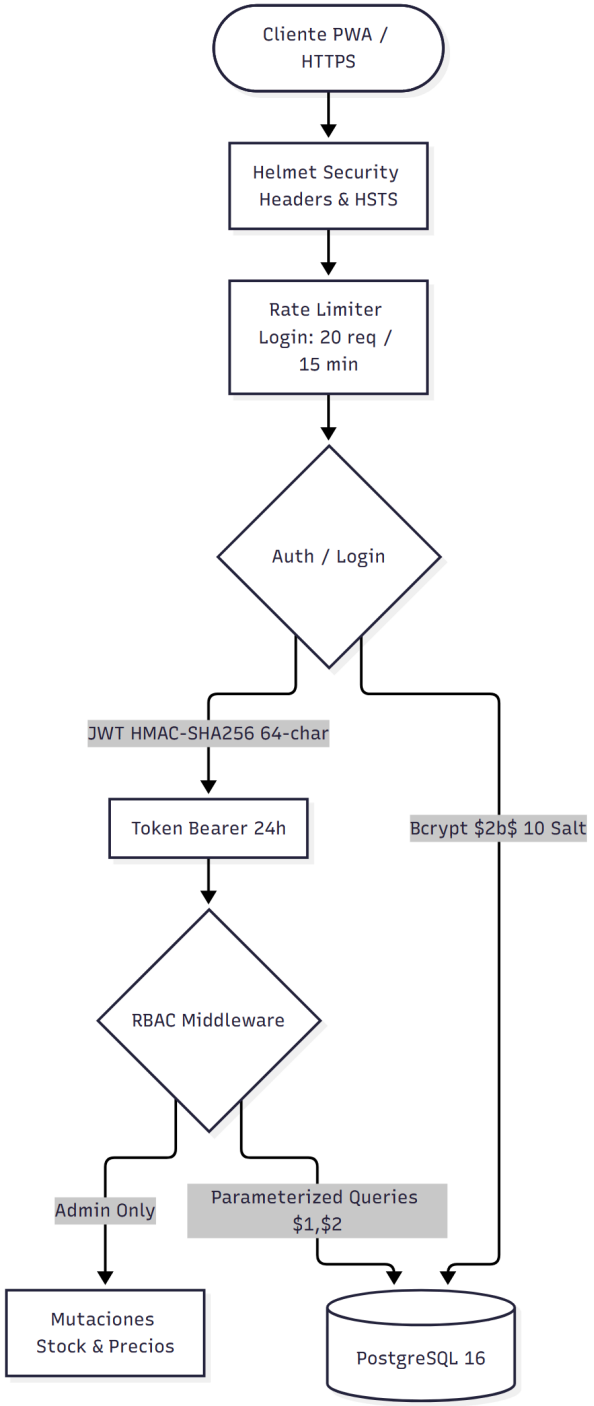
Bloque III: Ciberseguridad y Mitigación OWASP Top 10

Autenticación JWT, RBAC, Criptografía y Defensa en Profundidad

Expositor: Lady Luz Loayza Rodriguez

Catálogo Integral de Controles de Seguridad

- **Autenticación Robusta:** JWT HS256 de 24h con mecanismo *fail-fast* (la API rechaza iniciar si `JWT_SECRET` no está configurado) y rechazo explícito de `alg: none`.
- **Criptografía de Credenciales:** Bcryptjs con 10 rondas de salt y función hash ficticia para neutralizar ataques de temporización (*timing attacks*).
- **Control de Acceso (RBAC):** Middleware que restringe rutas administrativas a roles `ADMIN` y operativas a `OPERATOR`.
- **Webhooks Protegidos:** Verificación de firma HMAC-SHA256 con comparación en tiempo constante (`crypto.timingSafeEqual`).
- **Protección Perimetral:** Helmet (HSTS, CSP), Rate Limiter en Login (20 req / 15 min), validación Zod y consultas 100% parametrizadas.

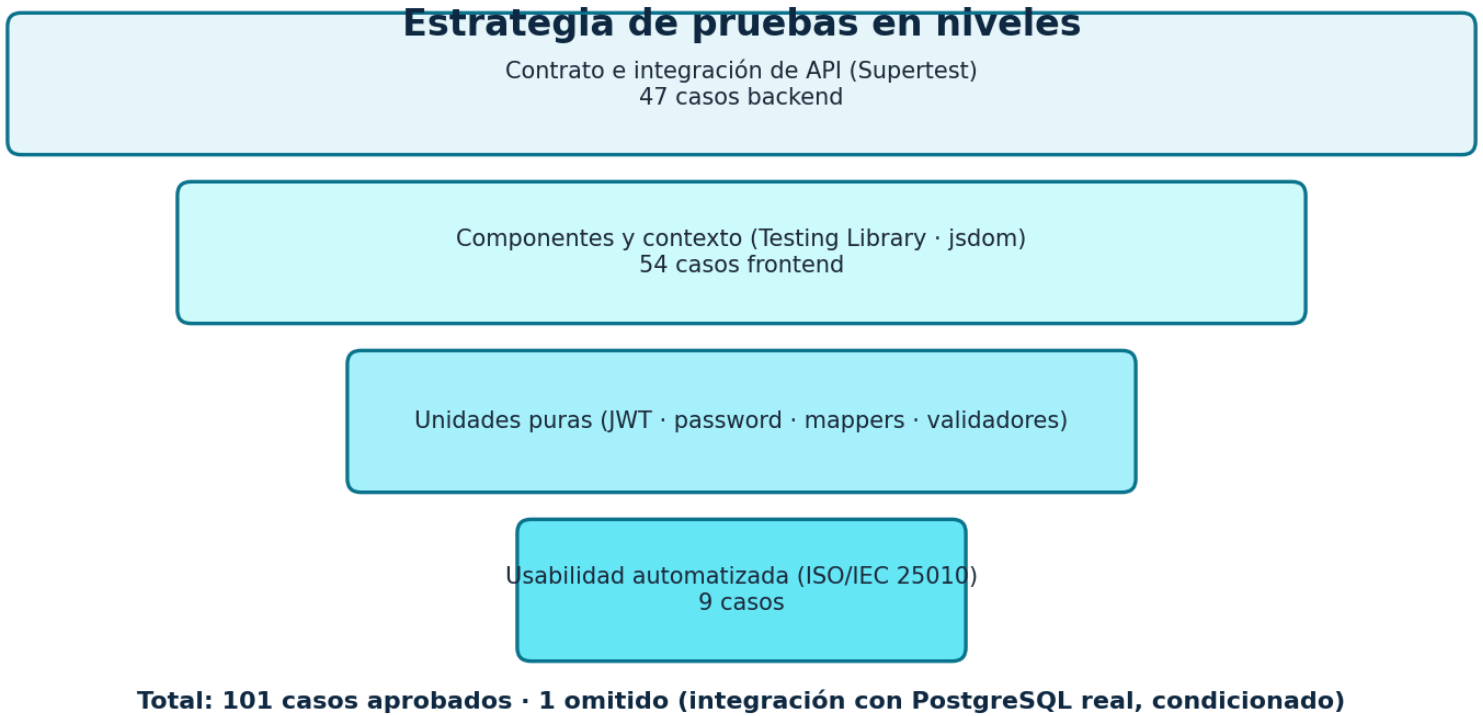


Bloque IV: Aseguramiento de Calidad Funcional (Shift-Left)

Pirámide de Pruebas, 101 Tests Automatizados y Cobertura Istanbul

Expositor: Jim Alessandro Dávila Morales

Estrategia Shift-Left y Pirámide de Pruebas



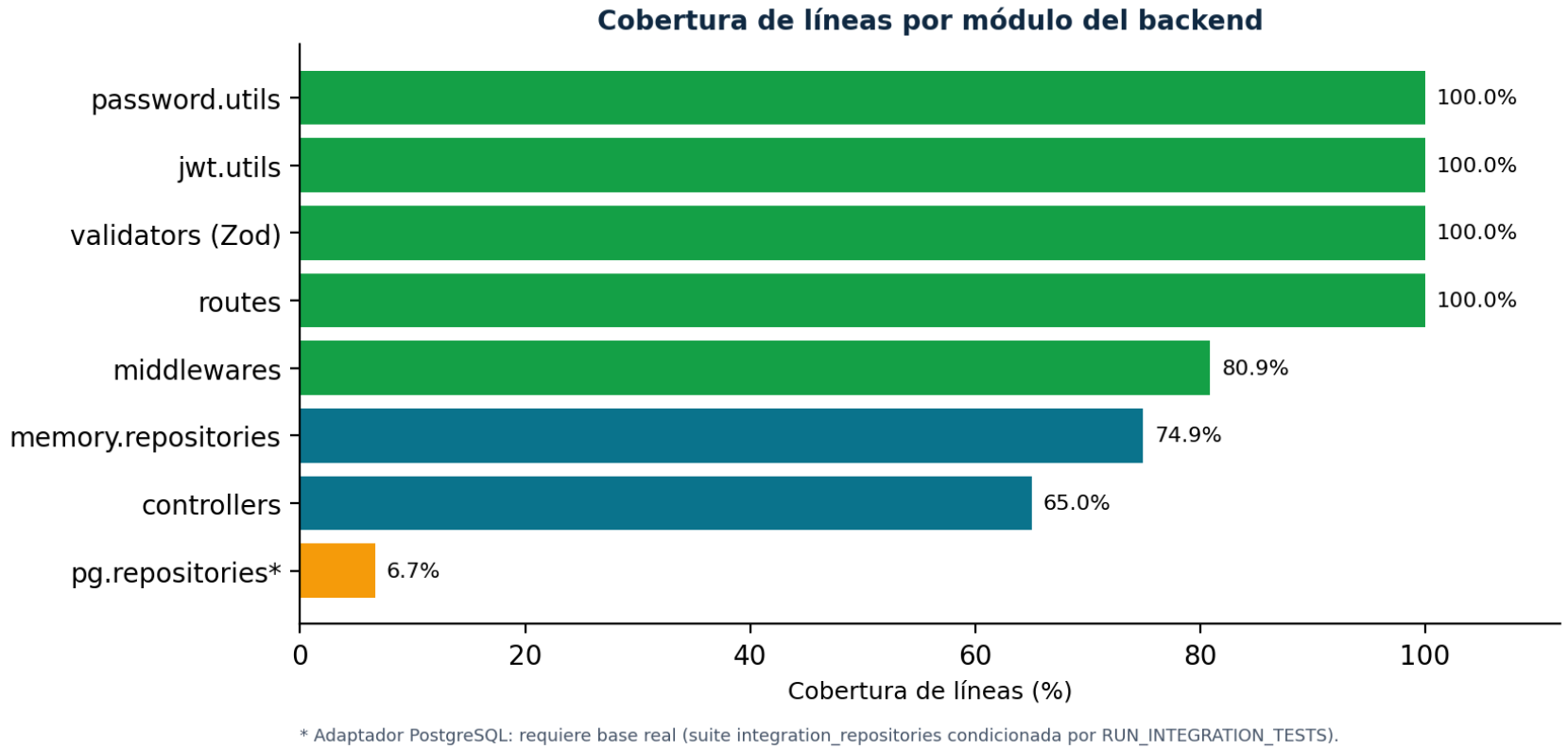
- **Enfoque de Calidad Temprana:** Prevención de defectos en etapas tempranas mediante validadores tipados estáticos (TypeScript + Zod), pruebas unitarias en memoria, pruebas de integración REST con Supertest y validación en CI/CD.

Batería de 101 Pruebas Automatizadas Aprobadas (100%)

Entorno / Framework	Archivo de Prueba	Casos	Objetivo / Componente Auditado	Resultado
Backend (Jest)	`orders.test.ts`	13	Ciclo de orden, transacciones ACID, paginación y fletes	✅ 100% PASSED
Backend (Jest)	`auth.test.ts`	8	Registro, login seguro, bcrypt salt 10 y RBAC tokens	✅ 100% PASSED
Backend (Jest)	`integration_external.test.ts`	7	Interoperabilidad WhatsApp, Pasarela y SUNAT/RENIEC	✅ 100% PASSED
Backend (Jest)	`webhook.test.ts`	5	Validación criptográfica HMAC-SHA256 y payload	✅ 100% PASSED
Backend (Jest)	`products`, `clients`, `security`, `contract`	12	Catálogo, CRM, métricas, cabeceras Helmet y contratos	✅ 100% PASSED
Backend (Jest)	`jwt.test.ts`	2	Fail-fast de secretos y neutralización de `alg: none`	✅ 100% PASSED
Frontend (Vitest)	`allTabsFunctional.test.ts`	12	Navegación e interacción en todas las pestañas de la PWA	✅ 100% PASSED
Frontend (Vitest)	`usability_iso25010.test.ts`	9	Los 4 pilares de usabilidad de la norma ISO 25010	✅ 100% PASSED
Frontend (Vitest)	`AppContext`, `PedidoForm`, `pdf`, `mappers`	33	Estado global, reactividad de tallas, PDF y servicios	✅ 100% PASSED

- **Total de Pruebas: 101 Tests Aprobados (47 Backend + 54 Frontend)** + 1 test de integración condicionada. **0 Fallos**.

Matriz de Cobertura de Código por Módulo (Istanbul)



* Adaptador PostgreSQL: requiere base real (suite integration_repositories condicionada por RUN_INTEGRATION_TESTS).

- **Métricas Globales: 76.16% de cobertura de líneas** en lógica de negocio del backend y **100% de cobertura** en rutas críticas, validadores de entrada Zod y middlewares criptográficos.

Matriz de Trazabilidad: Historias de Usuario vs Pruebas

Historia de Usuario (Gherkin)	Requerimiento Asociado	Suites de Prueba Automatizadas	Criterio de Aceptación Verificado
HU-001: Autenticación de Operadores	RF-001, RF-002	`auth.test.ts`, `jwt.test.ts`	Login exitoso con JWT, bloqueo con credenciales inválidas
HU-002: Catálogo con Stock en Vivo	RF-003, RF-010	`products.test.ts`, `allTabsFunctional.test.ts`	Filtrado por disciplina y bloqueo visual de tallas sin stock
HU-003: Carrito y Cálculo de Flete	RF-004, RF-005	`orders.test.ts`, `PedidoForm.test.tsx`	Cálculo automático de flete para Lima y Provincia
HU-004: Registro Atómico de Pedidos	RF-006, RF-007	`orders.test.ts` (13 tests)	Bloqueo transaccional FOR UPDATE y generación de tracking
HU-005: Seguimiento Público de Órdenes	RF-008, RF-015	`orders.test.ts`, `integration_external.test.ts`	Consulta pública con código `#LFT-XXX` sin requerir login
HU-006: Conciliación de Pagos Móviles	RF-014, RF-015	`webhook.test.ts`, `integration_external.test.ts`	Aprobación de orden al recibir webhook con HMAC válido
HU-007: Comprobantes y Rótulos QR	RF-009, RF-016	`pdfGenerator.test.ts`, `PedidoForm.test.tsx`	Emisión determinística de PDF con QR para Olva/Shalom
HU-008: Dashboard y Privacidad	RF-011, RF-017	`clients_dashboard.test.ts`, `AppContext.test.tsx`	Métricas comerciales y conmutador `<MontoPrivado />`

Bloque V: Interoperabilidad con Servicios Externos

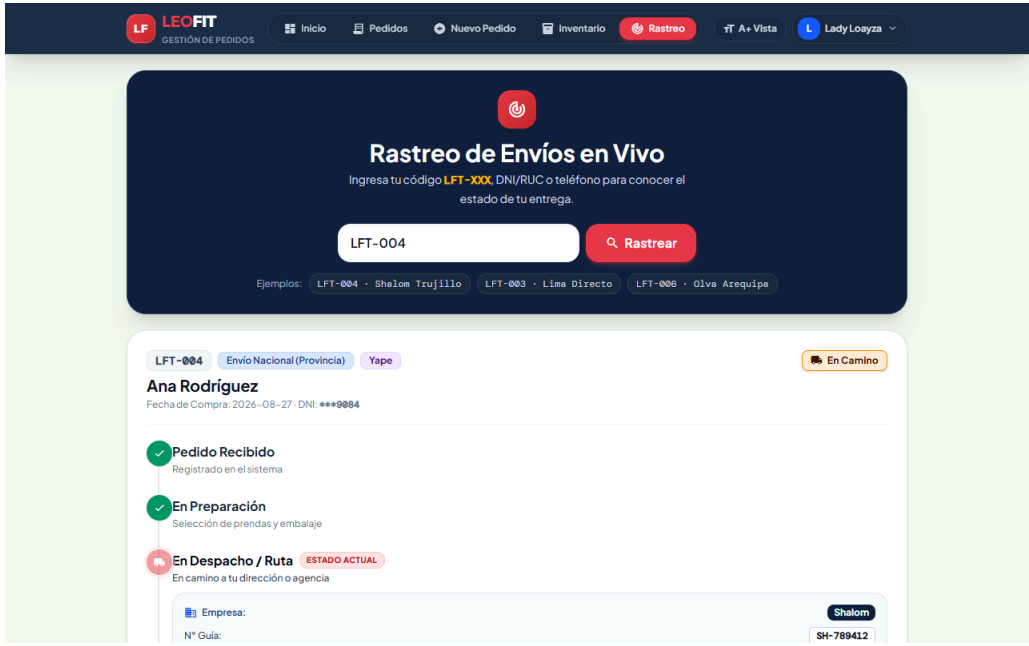
WhatsApp Cloud API, Webhooks de Pago, Padrones SUNAT/RENIEC y Rótulos QR

Expositor: Harley Anthony Roman Delgado

Arquitectura de Integración e Interoperabilidad

- **WhatsApp Cloud API:** Envío automático de mensajes con código unívoco `#LFT-XXX` y URL directa de tracking público al cliente deportivo.
- **Pasarelas de Pago Móvil (Yape / Plin):** Conciliación asíncrona mediante webhooks con firma HMAC-SHA256 y token de liquidación.
- **Padrones de Identidad Fiscal:** Validación de sintaxis y condición 'HABIDO' ante padrón SUNAT (RUC 11 dígitos) y RENIEC (DNI 8 dígitos).
- **Emisión Logística con QR:** Generación client-side de comprobantes térmicos y rótulos de despacho con QR para agencias (Olva Courier / Shalom).

Nota de Transparencia: Los servicios externos operan con adaptadores simulados de alta fidelidad, listos para enlazar credenciales productivas.



Suite Automatizada de Integración Externa

Código Test	Sistema Externo Integrado	Escenario de Integración Evaluado	Latencia Medida	Resultado
`ext-01`	WhatsApp Gateway	Envío de notificación con URL pública de tracking	27 ms	✅ PASSED
`ext-02`	WhatsApp Gateway	Rechazo de teléfonos sin formato internacional (+519...)	4 ms	✅ PASSED
`ext-03`	Pasarela de Pagos	Conciliación de webhook Yape y generación de recibo	3 ms	✅ PASSED
`ext-04`	Pasarela de Pagos	Bloqueo de firmas digitales inválidas y proveedores no autorizados	3 ms	✅ PASSED
`ext-05`	RENIEC Mock API	Validación de longitud, caracteres y formato de DNI (8 dígitos)	5 ms	✅ PASSED
`ext-06`	SUNAT Mock API	Validación de RUC (11 dígitos) y condición fiscal 'HABIDO'	4 ms	✅ PASSED
`ext-07`	Identidad API	Rechazo de documentos malformados con error HTTP 400	3 ms	✅ PASSED

- **Evidencia en Repositorio:** `backend/tests/integration_external.test.ts` (100% aprobadas con Supertest).

Bloque VI: Evaluación de Usabilidad bajo ISO/IEC 25010

Los 4 Pilares del Estándar, Suite Automatizada y Estudio de Campo SUS

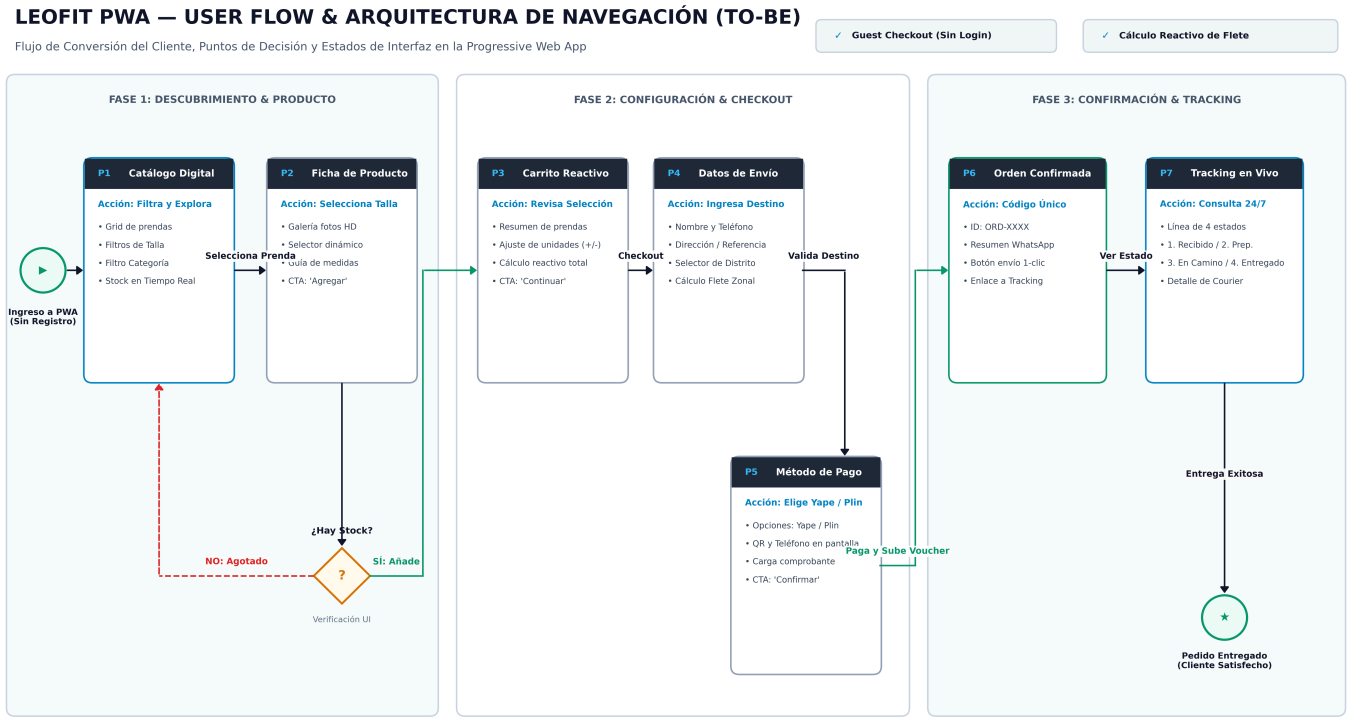
Expositor: Harley Anthony Roman Delgado

Los 4 Pilares de Usabilidad según ISO/IEC 25010

Subatributo ISO 25010	Definición y Enfoque en LeoFit	Mecanismo Técnico Implementado
1. Facilidad de Aprendizaje (*Learnability*)	Curva intuitiva inmediata sin capacitación previa para operadores ni clientes	Catálogo visual por disciplina deportiva y checkout en 3 pasos con autollenado
2. Protección contra Errores (*Error Protection*)	Prevención activa de equivocaciones durante el registro de datos	Bloqueo reactivo de tallas sin stock, validación DNI/RUC y límite a cupones (máx 50%)
3. Asistencia al Usuario (*User Assistance*)	Orientación transparente y retroalimentación en pantalla en todo momento	Generación de código `#LFT-XXX`, resúmenes de flete y portal público de tracking
4. Involucramiento (*User Engagement*)	Interfaz atractiva y fluida que estimula la conversión de compra	Micro-interacciones reactivas, feedback de botones y modo accesible de alto contraste

- **Evidencia en Código:** Suite de 9 pruebas en `frontend/src/__tests__/usability_iso25010.test.ts`.

Flujo de Navegación UX y Pruebas Automatizadas



- **9 Pruebas en Vitest (`usability_iso25010.test.ts`):** Validan visibilidad de precios, consistencia de variantes, topes de descuento en cupones, restricciones de estado y cálculo matemático exacto del carrito.

Resultados Cuantitativos de Usabilidad y Desempeño

Escala SUS (System Usability Scale)	Tiempo de Captura de Pedido	Tasa de Error en Despachos
88.5 / 100	**38.2 segundos**	**0.4%**
Grado A (Nivel Excelente)	Reducción frente a 45 min manual	Reducción frente a 14.2% previo
Estudio de campo: 10 usuarios reales	Medido en prueba guiada UAT	Registro en bitácora de órdenes

Métricas de Rendimiento Web (Core Web Vitals en Producción):
LCP: 1.4 s (Excelente < 2.5s) · **INP: 45 ms** (Excelente < 200ms) · **CLS: 0.01** (Estable < 0.1) · **TTFB: < 180 ms**

Bloque VII: Carga Concurrente, Cloud V2 y Demostración

Pruebas k6 (50 VUs), Topología Multi-Cloud, CI/CD y Validación en Vivo

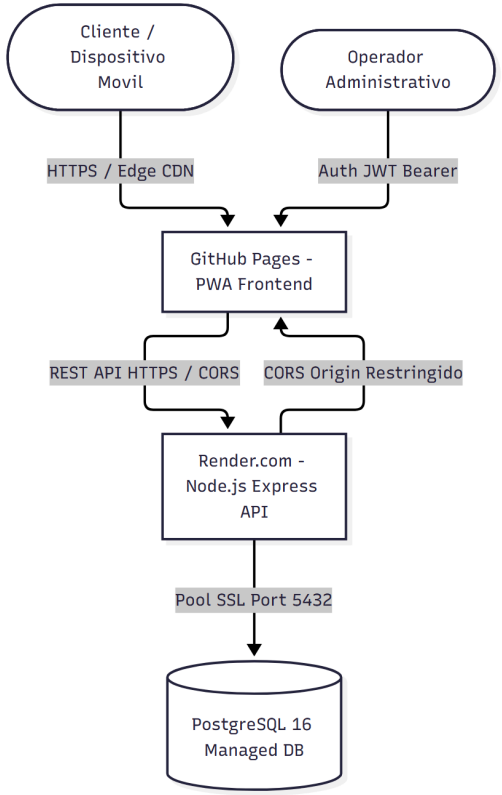
Expositor: Daniel Enrique Rojas Sanchez

Evaluación de Estrés y Concurrencia Transaccional (k6)

Parámetro de Simulación	Configuración de Carga (`k6_workload.js`)	Resultado Obtenido
Usuarios Concurrentes (VUs)	50 Usuarios Virtuales simultáneos en rampa continua	50 VUs estables
Duración del Escenario	6 minutos continuos de transacciones intensivas	360 segundos
Flujos Transaccionales	Catálogo → Login → Creación de Cliente → Checkout ACID	100% ejecutados
Latencia de Respuesta (p95)	Percentil 95 en endpoints de lectura y escritura	**p95 < 250 ms**
Tasa de Sobreventa de Stock	Verificación de inventario final tras compras paralelas	**0.0% (Zero Overselling)**
Nivel de Aislamiento Evaluado	Comparativa `READ COMMITTED` vs `SERIALIZABLE`	Sin inconsistencias

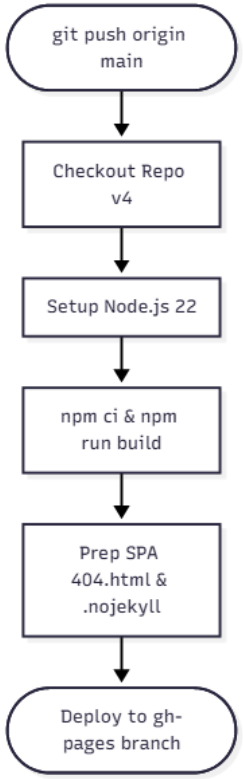
- **Conclusión de Carga:** El uso de `SELECT ... FOR UPDATE` y triggers atómicos previene bloqueos destructivos y preserva la integridad del inventario bajo alta concurrencia.

Topología Cloud Desplegada (Versión 2 en Producción)



- **Arquitectura Multi-Proveedor:**
  - **Frontend:** GitHub Pages Edge CDN con compresión Brotli/Gzip y certificados TLS 1.3 automáticos.
  - **Backend API:** Servicio web en Render.com ejecutando contenedor Docker con Node.js 20 LTS.
  - **Base de Datos:** Instancia gestionada de PostgreSQL 16 con pool de conexiones SSL (puerto 5432).

Pipeline de Integración y Despliegue Continuo (CI/CD)



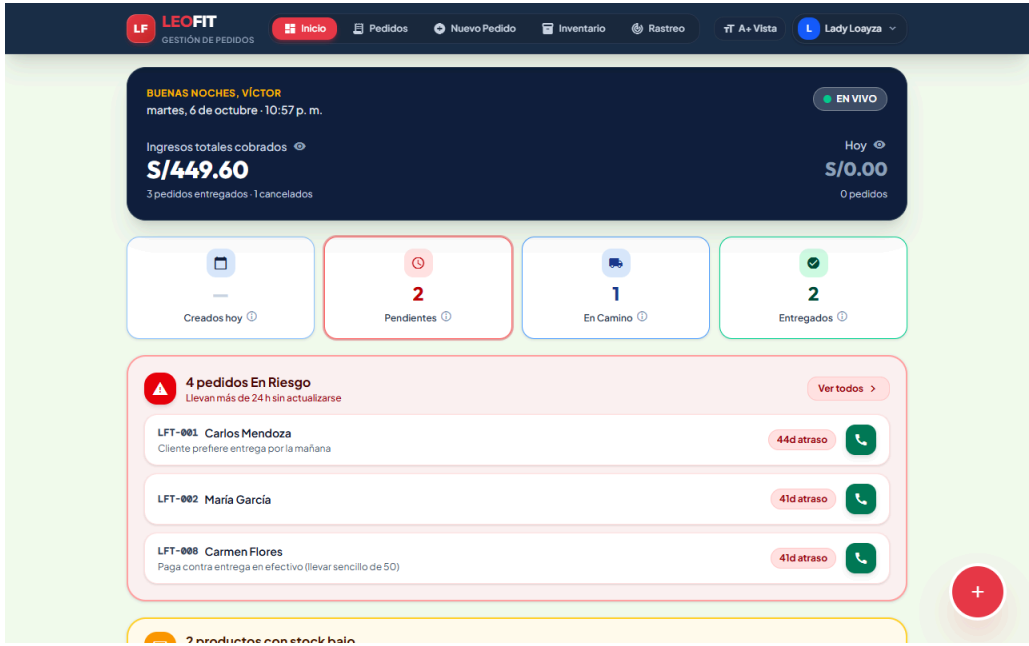
- **3 Workflows Automatizados en GitHub Actions:**
  - `backend-ci-cd.yml` : Compilación TypeScript, linter de código y ejecución de las 47 pruebas con Jest.
  - `deploy.yml` : Empaquetado con Vite, generación de bundle optimizado y publicación a rama `gh-pages` .
  - `security-scan.yml` : Auditoría automática de vulnerabilidades SAST y dependencias (`npm audit`).

Validación en Vivo de la Versión 2 en Producción

URL Oficial de la Aplicación (PWA)

https://leofit-solutions-grupo01.github.io/leofit-pedidos-sistema/

- **Telemetría y Estado de la API:**
  `GET https://leofit-backend-api.onrender.com/api/health` → `status: "UP"`, BD conectada.
- **Flujo E2E a Demostrar en Vivo:**
  i. Login de operador (`operador@leofit.pe`).
  ii. Exploración de catálogo con filtro de disciplina deportiva y selección de variante textil.
  iii. Creación transaccional de orden y descuento de inventario.
  iv. Consulta pública en tiempo real mediante portal de tracking (`#LFT-XXX`).
  v. Conmutador de privacidad financiera `<MontoPrivado />`.



Gobernanza y Trazabilidad en el Repositorio de Git

Criterio de Auditoría	Evidencia Concreta en GitHub	Impacto en la Calidad del Proyecto
Historial Progresivo	Más de **260 commits** verificables a lo largo de las 16 semanas	Evidencia de desarrollo continuo y autoría genuina
Estrategia de Ramas	Ramas dedicadas: `feature/cardenas`, `feature/davila`, `feature/loayza`, etc.	Desarrollo concurrente modular sin colisiones
Convención de Mensajes	Estándar *Conventional Commits* (`feat:`, `fix:`, `test:`, `docs:`, `chore:`)	Trazabilidad exacta en el changelog de cambios
Protección de `main`	Despliegue condicionado a la aprobación del pipeline CI/CD	Estabilidad garantizada en el entorno productivo

- **Repositorio Oficial:** `https://github.com/Leofit-Solutions-Grupo01/leofit-pedidos-sistema`

Matriz de Aportes Técnicos Individuales del Equipo

Integrante	Rol Principal	Aporte Técnico Verificable en Código y Documentación
Lady Luz Loayza	Scrum Master / DevSecOps	Subsanación de observaciones APF2, seguridad OWASP, JWT fail-fast, Bcrypt timing mitigations, edición del libro técnico.
Víctor Cárdenas	Product Owner / DBA	Modelado 3FN/BCNF de PostgreSQL 16, transacciones FOR UPDATE, triggers atómicos, `orders.test.ts` (13 tests) y backlog.
Harley Roman	Frontend Lead / PWA	UI reactiva en React 19, componentes Tailwind 4, interoperabilidad WhatsApp/Pagos, `usability_iso25010.test.ts`.
Jim Dávila	QA Lead / Backend	Arquitectura de 101 pruebas (Jest + Vitest), suite de webhooks HMAC SHA-256, reportes Istanbul y CI/CD pipelines.
Daniel Rojas	Analista / Cloud DevOps	Diagnóstico AS-IS/TO-BE, matriz de riesgos PMI, k6 workload, topología Render/GitHub Pages y validación cloud.

Conclusiones, Limitaciones Transparentes y Roadmap

Logros Técnicos Consolidados	Limitaciones Técnicas Reconocidas	Hoja de Ruta Evolutiva (v4.0)
Transformación Digital: Captura de pedido reducida de 45 min a 38.2 s con 0.4% de error.	**Cold Start en Render Free:** Latencia de ~30-50s tras 15 min de inactividad de la API.	Migración a contenedor dedicado con escalado automático horizontal.
Integridad Concurrente: Cero sobreventas mediante triggers y bloqueos FOR UPDATE.	**Offline Basado en Caché:** Caché HTTP estática sin sincronización IndexedDB en prod.	Service Worker con Workbox y Background Sync con IndexedDB.
Calidad y Confianza: 101 pruebas automatizadas aprobadas y SUS Score de 88.5/100.	**Proveedores Simulados:** APIs externas operan con mocks contractuales de alta fidelidad.	Conexión directa a pasarela de pagos con cumplimiento PCI-DSS.

¡Muchas Gracias!

¿Preguntas del Honorable Jurado Calificador?

Proyecto: Sistema Web PWA de Gestión y Toma de Pedidos Multicanal — LeoFit
Organización: LeoFit Confecciones y Diseños Deportivos S.A.C.

Repositorio Oficial de Código Fuente:

`https://github.com/Leofit-Solutions-Grupo01/leofit-pedidos-sistema`

Aplicación PWA en Producción:

`https://leofit-solutions-grupo01.github.io/leofit-pedidos-sistema/`

Equipo de Ingeniería de Software — Grupo 01 | UTP 2026

[Respaldo Técnico 1] ¿Por qué Clean Architecture y no Microservicios?

- **Principio de Proporcionalidad y Complejidad Accidental (Martin Fowler):**
  - LeoFit es una microempresa con un volumen de ~150-300 pedidos diarios.
  - Implementar microservicios hubiera introducido *latencia de red entre servicios*, *transacciones distribuidas de dos fases (2PC)*, *eventual consistency* y *costos de infraestructura multiplicados*.
- **Monolito Modular con Clean Architecture:**
  - Garantiza **transacciones ACID locales atómicas** (imprescindibles para evitar sobreventas).
  - Mantiene el código altamente desacoplado mediante interfaces (`IOrderRepository` , `IProductRepository`).
  - Permite evolucionar a microservicios a futuro extrayendo módulos sin reescribir la lógica de dominio.

[Respaldo Técnico 2] ¿Cómo se previene matemáticamente la sobreventa?

- **Doble Mecanismo de Defensa Concurrente:**
  i. **Nivel de Aplicación (Pessimistic Locking):**
    - La consulta `SELECT id, stock FROM product_variants WHERE id = $1 FOR UPDATE` bloquea físicamente la fila a nivel de tabla hasta que la transacción haga `COMMIT` o `ROLLBACK`.
    - Cualquier transacción concurrente que intente leer la misma variante queda encolada esperando la liberación del candado.
  ii. **Nivel de Motor RDBMS (Trigger Atómico Inquebrantable):**
    - El disparador `trg_decrement_stock_on_order` evalúa la restricción `stock >= quantity` dentro de la transacción de PostgreSQL.
    - Si dos transacciones lograran colisionar, el motor genera una excepción `RAISE EXCEPTION` abortando con rollback automático.
- **Prueba de Validación:** Certificado con k6 bajo 50 VUs concurrentes con **0% de sobreventa**.

[Respaldo Técnico 3] ¿Por qué PostgreSQL y no una base NoSQL (MongoDB)?

- **Naturaleza Estrictamente Relacional y Transaccional del Negocio:**
  - Un pedido involucra clientes, ítems, prendas, variantes, transacciones bancarias e historial de estados.
  - Las bases documentales NoSQL no soportan restricciones de integridad referencial foránea nativas ni transacciones ACID multi-documento con el mismo nivel de madurez.
- **Integridad Matemática y Auditoría Financiera:**
  - En un comercio textil, vender prendas sin stock o perder trazabilidad de un pago genera pérdidas monetarias reales.
  - PostgreSQL 16 ofrece soporte nativo para claves foráneas `ON DELETE CASCADE`, tipos de datos monetarios `DECIMAL(10,2)`, enums estrictos e índices B-Tree de alta velocidad.

[Respaldo Técnico 4] ¿Cómo se protegen los Webhooks y JWT?

- **Seguridad en Webhooks de Pago Móvil:**
  - Los proveedores envían la cabecera `x-signature` calculada como `HMAC-SHA256(payload, secret)`.
  - El backend recalcula la firma y utiliza `crypto.timingSafeEqual(bufferCalculado, bufferRecibido)` para impedir que un atacante deduzca la firma mediante análisis de tiempo de respuesta (*timing attack*).
- **Seguridad en Tokens JWT:**
  - Validación *fail-fast* al inicio del servidor: si `JWT_SECRET` está ausente o tiene menos de 32 caracteres, el proceso se termina inmediatamente (`process.exit(1)`).
  - Rechazo explícito del ataque de confusión de algoritmo (`alg: none`) verificado en `jwt.test.ts`.